

CSA1717-ARTIFICIAL INTELLIGENCE

CO2-AT3-DRY RUN TEST

SOLUTION

QUESTION 1 – DETAILED SOLUTION

*Solution: A Search Algorithm**

The A* (A-Star) Search Algorithm is one of the most efficient informed search algorithms used in Artificial Intelligence for finding the shortest path between a source node and a destination node. Unlike uninformed search algorithms such as Breadth-First Search (BFS) or Depth-First Search (DFS), A* makes use of heuristic information to estimate the distance from the current node to the goal node. This heuristic guidance enables the algorithm to reduce unnecessary exploration and quickly identify the optimal path.

The algorithm evaluates every node using the evaluation function

$$f(n) = g(n) + h(n)$$

where:

- **g(n)** = Actual cost from the start node to the current node.
- **h(n)** = Estimated (heuristic) cost from the current node to the goal node.
- **f(n)** = Total estimated cost of the path through the current node.

For this problem, the weighted graph consists of six nodes: **A, B, C, D, E, and G**. The heuristic values are:

Node h(n)

A 7

B 6

C 4

D 3

E 2

Node $h(n)$

G 0

The objective is to determine the shortest path from **A** (Start Node) to **G** (Goal Node).

Initial State

Initially, the Open List contains only the start node **A**, while the Closed List is empty.

Open List: A

Closed List: Empty

For node A,

$$g(A)=0$$

$$h(A)=7$$

Therefore,

$$f(A)=0+7=7$$

Since A is the only node available, it is selected for expansion.

Iteration 1

The current node is **A**.

The neighboring nodes of A are:

- B with edge cost 2
- C with edge cost 4

The algorithm calculates:

For node B

$$g(B)=0+2=2$$

$$h(B)=6$$

$$f(B)=2+6=8$$

For node C

$$g(C)=0+4=4$$

$$h(C)=4$$

$$f(C)=4+4=8$$

Both nodes are inserted into the Open List.

Open List:

B(8), C(8)

Closed List:

A

Although both have the same evaluation value, B is selected first because it was inserted earlier.

Iteration 2

Current node = **B**

The neighbors of B are:

- C
- D
- E

Node C

New cost

$$g(C)=2+3=5$$

Current best cost for C is already 4.

Since $5 > 4$, no update is made.

Node D

$$g(D)=2+7=9$$

$$h(D)=3$$

$$f(D)=9+3=12$$

Node E

$$g(E)=2+2=4$$

$$h(E)=2$$

$$f(E)=4+2=6$$

Open List becomes

E(6)

C(8)

D(12)

Closed List

A

B

The node with the minimum evaluation function is **E**.

Iteration 3

Current node = **E**

Neighbors are:

- B
- C
- D

Node B is already explored.

Node C

New cost

$$g(C)=4+3=7$$

Existing cost is 4.

Hence no update.

Node D

New cost

$$g(D)=4+2=6$$

Previous cost of D was 9.

Since 6 is smaller,

update D

$$h(D)=3$$

$$f(D)=6+3=9$$

Open List

C(8)

D(9)

Closed List

A

B

E

The node with minimum $f(n)$ becomes C.

Iteration 4

Current node = C

Neighbors:

- A
- B
- E

All neighboring nodes have already been explored or have better costs.

Therefore, no changes occur.

Open List

D(9)

Closed List

A

B

C

E

Iteration 5

Current node = **D**

Neighbors:

- B
- E
- G

Node G is reached.

$$g(G)=6+2=8$$

$$h(G)=0$$

$$f(G)=8$$

Open List

G(8)

Closed List

A

B

C

D

E

Iteration 6

Current node = **G**

Since G is the goal node, the search terminates.

Optimal Path

The final shortest path is

$A \rightarrow B \rightarrow E \rightarrow D \rightarrow G$

Path Cost Calculation

$A \rightarrow B = 2$

$B \rightarrow E = 2$

$E \rightarrow D = 2$

$D \rightarrow G = 2$

Total Cost

$= 2 + 2 + 2 + 2$

$= 8$

Explanation of Open List and Closed List

During every iteration:

- The **Open List** stores all nodes that have been discovered but not yet expanded.
- The **Closed List** stores nodes that have already been completely explored.

The algorithm always selects the node with the smallest **f(n)** from the Open List, ensuring that the most promising path is explored first.

Analysis

The heuristic values gradually decrease as the search approaches the goal node. Since the heuristic is admissible, the A* algorithm guarantees the shortest path. Compared with uninformed search methods, fewer nodes are expanded, making the search faster and more efficient.

Final Result

Optimal Path:

A → B → E → D → G

Total Path Cost:

8

Thus, the A* Search Algorithm successfully identifies the shortest path while minimizing unnecessary exploration through heuristic guidance.

QUESTION 2 – DETAILED SOLUTION

Solution: Minimax Algorithm with Alpha-Beta Pruning

The Minimax Algorithm is an adversarial search algorithm used in Artificial Intelligence to determine the optimal move in two-player, zero-sum games such as Chess, Tic-Tac-Toe, and Checkers. It assumes that both players make the best possible moves. The **MAX** player attempts to maximize the utility value, while the **MIN** player attempts to minimize it.

To improve efficiency, **Alpha-Beta Pruning** is applied. Alpha-Beta Pruning reduces the number of nodes evaluated by eliminating branches that cannot influence the final decision.

Initially,

Alpha (α)

$= -\infty$

Beta (β)

$= +\infty$

The given game tree consists of one MAX node, two MIN nodes, and six terminal nodes.

Step 1 – Evaluate Left MIN Node

Leaf values:

3

5

6

The MIN player chooses the minimum value.

Comparisons:

$\min(3)=3$

$\min(3,5)=3$

$\min(3,5,6)=3$

Selected Value

$=3$

The value 3 is returned to the MAX node.

The MAX node updates

$\alpha=\max(-\infty,3)$

$=3$

Step 2 – Evaluate Right MIN Node

Leaf values:

9

1

2

Initially

$$\beta = \infty$$

Evaluate 9

Minimum

$$= 9$$

$$\beta = \min(\infty, 9)$$

$$= 9$$

Since

$$\beta(9) > \alpha(3)$$

Continue.

Evaluate 1

Minimum

$$= \min(9, 1)$$

$$= 1$$

$$\beta = \min(9, 1)$$

$$= 1$$

Now

$$\beta = 1$$

$$\alpha = 3$$

Since

$$\beta \leq \alpha$$

$$1 \leq 3$$

the Alpha-Beta Pruning condition is satisfied.

Therefore, the remaining node with utility value **2** is **not evaluated** because it cannot change the outcome.

Pruning

The node containing utility value **2** is pruned.

This reduces computation while preserving the correct minimax result.

Values Returned

Left MIN

=3

Right MIN

=1

MAX Node Evaluation

MAX chooses

$\max(3,1)$

=3

Final Minimax Value

3

Best Move

The MAX player selects the **left subtree**, because it provides the highest utility value.

Alpha-Beta Updates

Step	Alpha Beta Action		
Start	$-\infty$	$+\infty$	Initialize
Left MIN	$-\infty$	3	Return 3

Step	Alpha	Beta	Action
MAX	3	$+\infty$	Update α
Right MIN (9)	3	9	Continue
Right MIN (1)	3	1	$\beta \leq \alpha$
Remaining Node Pruned -			Stop Evaluation

Analysis

Without Alpha-Beta Pruning, all **6 leaf nodes** would be evaluated.

With Alpha-Beta Pruning, only **5 leaf nodes** are evaluated because one node is pruned.

This reduces the computational effort and increases the efficiency of the Minimax algorithm, especially in large game trees.

Final Result

Best Move:

Left Subtree

Final Minimax Value:

3

Pruned Node:

2

Hence, the Minimax Algorithm with Alpha-Beta Pruning successfully determines the optimal move for the MAX player while reducing unnecessary computations by pruning branches that cannot affect the final outcome.